Name	Last commit message	Last commit date
..		
data	Картинки распределены по папкам	2 years ago
img	Картинки распределены по папкам	2 years ago
README.md	Картинки распределены по папкам	2 years ago

README.md

Наивный байесовский классификатор

Цель работы

Научиться применять модели наивного байесовского классификатора с использованием библиотеки sklearn для решения задач машинного обучения.

Содержание работы

1. Загрузите датасет *breast_cancer* из библиотеки *sklearn*.
2. Выведите в виде гистограммы распределения непрерывных атрибутов в этом наборе данных.
3. Постройте модель наивного байесовского классификатора на первых двух столбцах и оцените ее качество.
4. Постройте ту же модель на полном датасете и сравните ее качество с первой.
5. Отберите признаки, соответствующие выбранному модельному виду распределения. Постройте модель на выбранных признаках и сравните ее с предыдущей.

Методические указания

Загрузим встроенный в библиотеку *sklearn* датасет с данными о диагностике рака груди:

```
data = datasets.load_breast_cancer()
```

Если вы встречаете этот набор данных впервые, самостоятельно выведите его описание и познакомьтесь с его структурой. Мы же выделим целевую переменную и входные атрибуты. Для целей визуализации выберем первые два столбца:

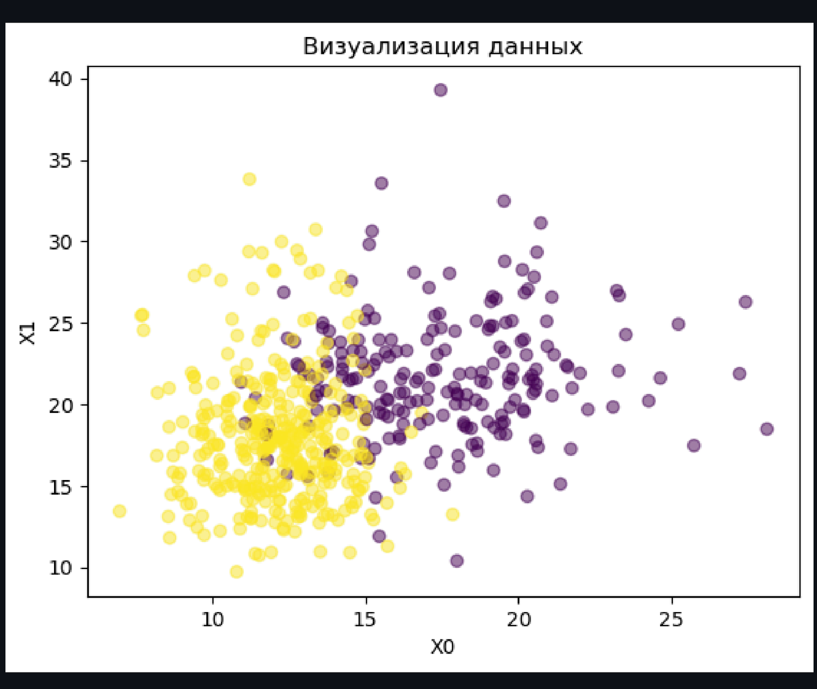
```
X = data.data[:,:2]
y = data.target
```

Для работы байесовского классификатора важно понимать, какие значения может принимать целевая переменная. Поэтому выведем информацию о количестве разных значений целевой переменной в получившейся обучающей выборке:

```
df = pd.DataFrame(data.data[:,:2], columns=data.feature_names[:2])
df['target'] = data.target
df['target'].value_counts()
```

```
target
1    357
0    212
Name: count, dtype: int64
```

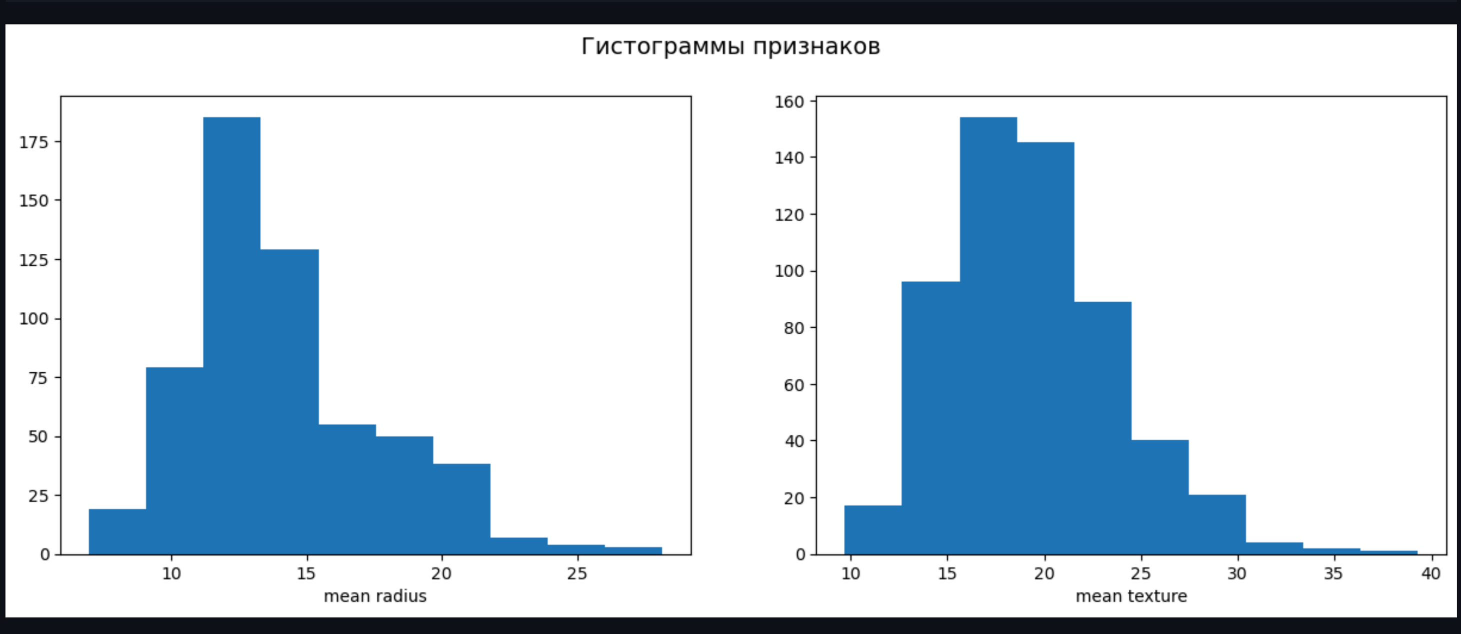
Теперь можно визуализировать получившееся распределение точек:



Мы видим, что данные являются линейно неразделимыми. Да и вообще, классы довольно сильно перемешаны. Посмотрим, что может сделать классификатор на основе формуля Байеса.

Для создания модели наивного байесовского классификатора необходимо выбрать то, каким именно распределением модель будет аппроксимировать функцию плотности вероятности непрерывных переменных. Так как обе наши входные переменные как раз непрерывные, лучш евывести форму их распределения, чтобы понять, на какое известное статистическое распределение она похожа. Для этого воспользуемся инструментом построения гистограмм библиотеки *matplotlib*. Построим сразу обе гистограммы (для двух атрибутов) на одном графике в подобластях:

```
plt.figure(figsize=(15, 5))
for i in range(2):
    plt.subplot(1, 2, i+1)
    plt.hist(X[:, i])
    plt.xlabel(data.feature_names[i])
plt.suptitle('Гистограммы признаков',fontsize=14)
plt.show()
```



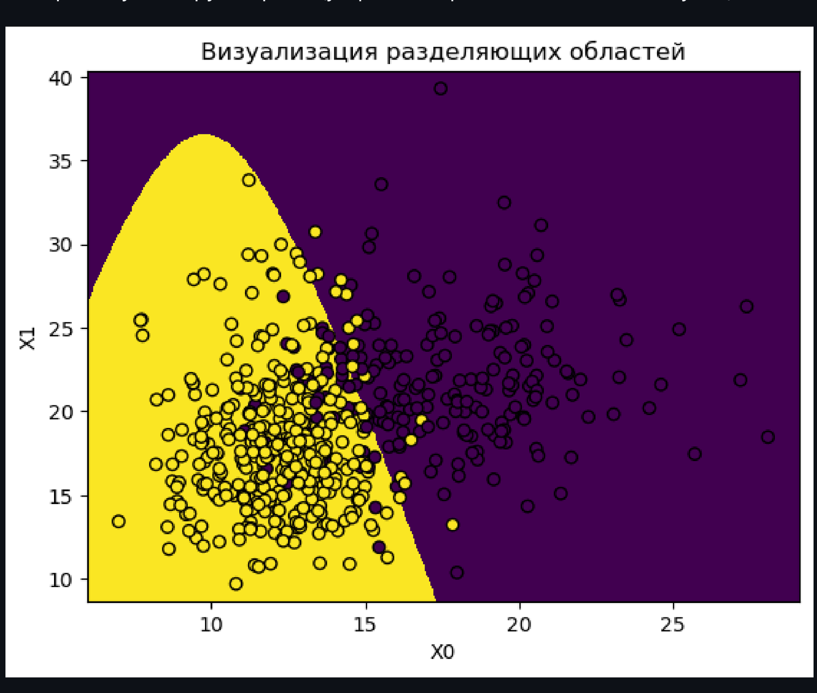
Из графиков видно, что форма распределения обоих признаков напоминает нормальное. Можно принять гипотезу о нормальности распределения наших признаков и использовать модель наивного Байеса с гауссовой аппроксимацией. Для создания можели инстанцируем соответствующих класс:

```
gaussian_nb = GaussianNB()
```

Обучим модель на имеющихся у нас данных:

```
gaussian_nb.fit(X, y)
```

Теперь визуализируем границу принятия решения и соответствующие области классификации:



Для более точной оценки эффективности модели посчитаем метрики. Для этого рассчитаем теоретические значения целевой переменной для объектов обучающей выборки:

```
y_pred = gaussian_nb.predict(X)
```

Тперерь выведем значения метрик. Мы можем здесь использовать какие угодно метрики эффективности моделей классификации, но для первого анализа будет достаточно вывести, например, матрицу классификации, точность и метрику F1:

```
print(confusion_matrix(y, y_pred))
print('Accuracy= ', accuracy_score(y, y_pred))
print('F1_score= ', f1_score(y, y_pred))
```

Мы получили, что модель правильно классифицирует 88,6% объектов обучающей выборки. Это при том, что мы обучались только на первых двух столбцах.

Естественно, нужно проверить, как наша модель будет работать при использовании всех данных, имеющихся в исходном датасете. Выделим целевую переменную и атрибуты снова:

```
X = data.data
y = data.target
X.shape, y.shape
```

Обучим модель на всех данных и сразу выведем те же самые метрики:

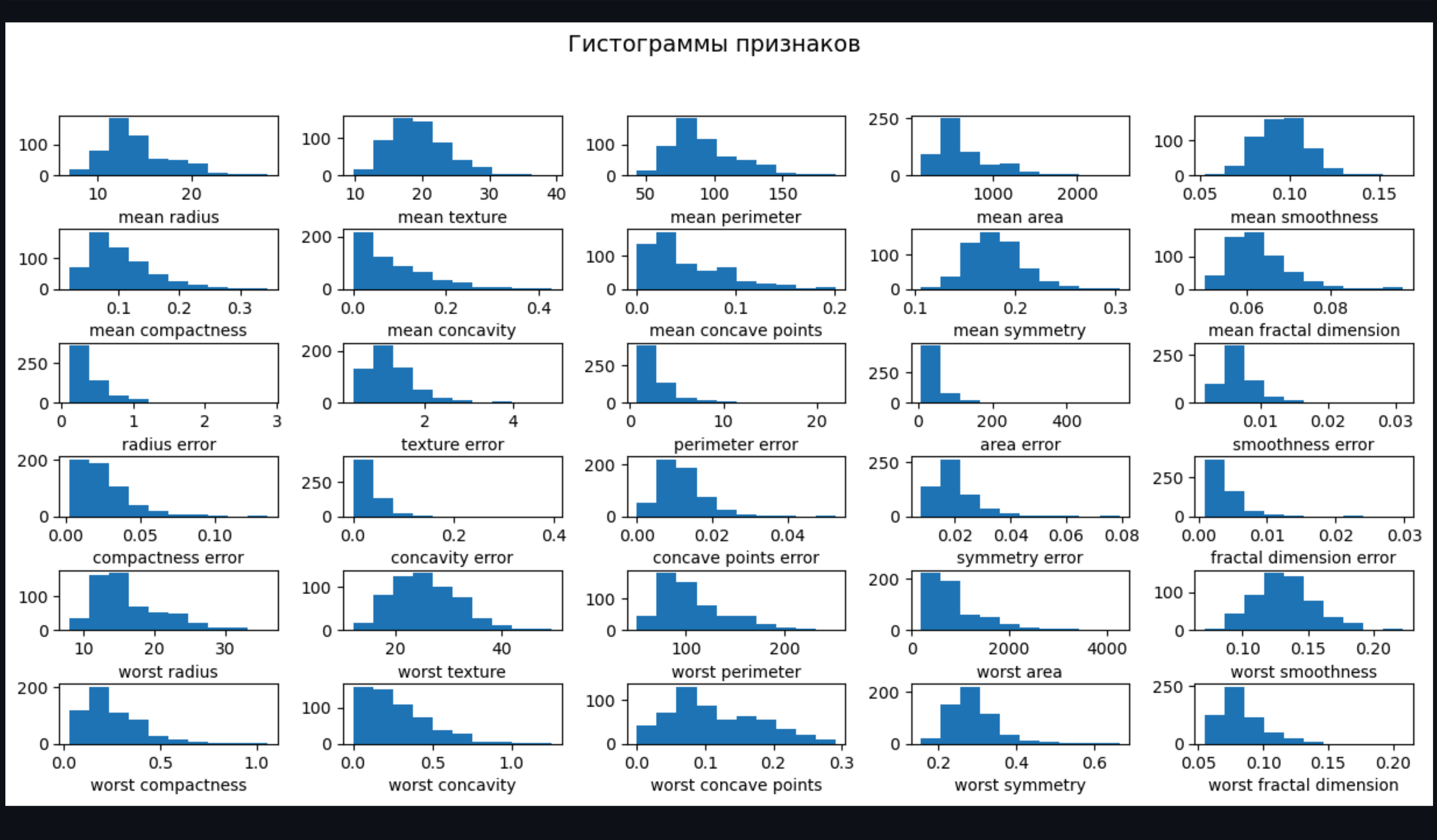
```
gaussian_nb.fit(X, y)
y_pred = gaussian_nb.predict(X)
print(confusion_matrix(y, y_pred))
print('Accuracy= ', accuracy_score(y, y_pred))
print('F1_score= ', f1_score(y, y_pred))
```

У нас получилась точность примерно 94,2%. Это значительно лучше, чем у первой модели, что естественно, так как мы используем гораздо больше информации об объектах обучающей выборки.

Так как использование байесовской модели тесно связано с аппроксимацией непрерывных расрпеделений, давайте выведем гистограммы всех атрибутов, которые мы использовали во второй модели:

```
f = plt.figure(figsize=(15, 7))
for i in range(30):
    plt.subplot(6, 5, i+1)
    plt.hist(X[:, i])

    plt.xlabel(data.feature_names[i])
f.subplots_adjust(hspace=0.9,wspace=0.3)
plt.suptitle('Гистограммы признаков',fontsize=14)
plt.show()
```



Из данного графика видно, что у большинства признаков распределение и правда напоминает нормальное. Но некоторые распределения совсем на него не похожи. Давайте удалим из набора данных те признаки, которые по своему виду распределения сильно отличаются от нормального:

```
df = df.drop(['mean concavity','radius error',
             'perimeter error', 'area error',
             'compactness error', 'concavity error',
             'fractal dimension error', 'worst area',
             'worst concavity'],
            axis=1)
df.head()
```

Обучим модель на оставшихся данных и оценим ее качество:

```
gaussian_nb.fit(X, y)
y_pred = gaussian_nb.predict(X)
print(confusion_matrix(y, y_pred))
print('Accuracy= ', accuracy_score(y, y_pred))
print('F1_score= ', f1_score(y, y_pred))
```

Мы получили, что такая урезанная модель дает даже чуть большую точность - около 94,6%. Это парадоксальный результат, так как мы использовали меньше информации. Но такое увеличение точности произошло потому, что мы исключили признаки, которые сильно не соответствуют данному виду модели. Таким образом можно сделать вывод, что такие принаки, будучи введенными в модель не способны добавить точности предсказания, а скорее "запутывают" классификатор.

Задания для самостоятельного выполнения

1. На использованном в работе наборе данных примените другие вариации модели наивного Байеса - Мультономиальный, Бернулли, категориальные и комплементарный. Для каждой модели сделайте вывод о ее применимости.
2. Загрузите набор данных о выживших на титанике, прилагающийся к этой работе. Повторите на нем моделирование из методических указаний. Попробуйте разные варианты байесовского классификатора в зависимости от форм расрпеделния эмпирических данных.
3. Загрузите набор данных о Титанике с сайта Kaggle. Обратите внимание на обилие категориальных переменных. Примените на нем наивный байесовский классификатор.

Контрольные вопросы

1. Какие условия должны выполняться в данных, чтобы можно было применять наивную байесовскую модель?
2. Какие виды непрерывных распределений существуют и наиболее распространены?
3. Как определить вид распределения численной переменной в датасете?
4. Как наивная байесовская модель работает с категориальными признаками?
5. Зачем при применении наивного байесовского классификатора может понадобится анализировать форму эмпирического распределения признаков?
6. В чем особенность модели категориального наивного Байеса?

Дополнительные задания

1. Напишите универсальную функцию рисования произвольного количества гистограмм, которую можно применить к любому датасету.
2. Используйте проверку статистических гипотез о виде распределения, чтобы выбирать вид непрерывного распределения более строго.
3. Автоматизируйте выбор вида непрерывного распределения.